

SYSTEM DESIGN INTERVIEW

Design a Distributed Rate Limiter

API gateway throttling at Stripe / Cloudflare scale

~ 40 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Statement
- 02** · Requirements
- 03** · Capacity Estimation
- 04** · High-Level Architecture
- 05** · Algorithm Comparison
- 06** · Token Bucket Deep Dive
- 07** · Sliding Window Counter
- 08** · Distributed Implementation
- 09** · Identity Hierarchy & Tiers
- 10** · Persistence Schema

- [11](#) · [Hot Keys & Sharding](#)
- [12](#) · [Failure Modes](#)
- [13](#) · [Observability & Abuse Detection](#)
- [14](#) · [Design Trade-offs](#)
- [15](#) · [Interview Playbook](#)

01

Problem Statement

What we're building and why

Design a **distributed rate limiter** that throttles API traffic in front of services such as **Stripe**, **Cloudflare**, or a **public API gateway**. It must enforce per-identity quotas across a fleet of stateless gateways, decide accept-or-reject in well under a millisecond, and never become the bottleneck for legitimate traffic.

Clarifying Questions

Question	Answer
Where does it live?	Sidecar/library inside the API Gateway tier; centralized counter store
What identifies a caller?	Tiered: API key > user_id > device > IP
How many quotas?	Per identity × endpoint × window (e.g., 1K rps, 10K/min, 1M/day)
Hard or soft limits?	Both: hard 429 by default; soft = log-only for canaries
Burst tolerance?	Yes — token bucket allows short bursts up to 2× steady rate
Failure policy?	Fail-open for read traffic, fail-closed for write/billing
Global vs regional?	Regional cluster of Redis; eventual cross-region reconciliation
Tiers?	Free / Premium / Enterprise with configurable quotas

INSIGHT

Why this matters

A rate limiter is on the hot path of every request — every API call pays its latency tax. A bad design slows the whole product; a good one is invisible until an abuser shows up.

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Allow / deny	Return ALLOW (200/202) or DENY (429 Too Many Requests)
Multi-window	Per-second, per-minute, per-hour, per-day limits coexist
Multi-identity	Match the tightest of: API key, user_id, IP, device fingerprint
Tiered quotas	Free / Premium / Enterprise with hot-reloadable config
Headers	Emit X-RateLimit-Limit, -Remaining, -Reset, Retry-After
Bypass list	Internal services and health checks skip the limiter
Observability	Per-key counters, allow/deny ratio, top-N abusers

Non-Functional Requirements

Requirement	Target
Throughput	10M requests/sec across the cluster
Added latency	p99 < 1 ms on the gateway critical path
Accuracy	± a few percent under burst; exact within a single node
Availability	99.99% — degrade to fail-open on store outage
Counter cardinality	~100M unique keys (user × endpoint × window)
State storage	~10 GB Redis cluster (replicated, 6+ shards)
Config propagation	< 5 sec for limit changes to reach all gateways

KEY

Headline Numbers

10M req/sec · **p99 < 1 ms** added · **~100M counter keys** · **~10 GB Redis** · fail-open reads / fail-closed writes.

03

Capacity Estimation

Sizing the counter store and gateway tier

Traffic

- Aggregate cluster traffic: **10,000,000 req/sec** across regions
- Per-gateway throughput: $\sim 50\text{K rps} \times 200 \text{ gateways} = 10\text{M rps}$
- Limiter calls per request: **1** (one bucket lookup per identity tier)
- Per-Redis-shard QPS: $10\text{M} / 16 \text{ shards} \approx \mathbf{625\text{K ops/sec}}$ (well within Redis limits)

Counter Cardinality

- Active users: $\sim 10\text{M}$ monthly, $\sim 1\text{M}$ concurrent
- Endpoints per service: ~ 20 distinct endpoints with their own quotas
- Windows per limit: per-second + per-minute + per-day = 3 windows
- Unique keys: $\sim 1\text{M}$ concurrent users $\times 20$ endpoints $\times 3$ windows $\approx \mathbf{\sim 60\text{M keys}}$
- Plus IP-based and API-key buckets: total **$\sim 100\text{M}$ unique keys**

Storage

- Per key: $\sim 64 \text{ B key string} + 16 \text{ B value (count + TTL metadata)} + \sim 20 \text{ B Redis overhead} \approx \mathbf{\sim 100 \text{ B}}$
- Total: $100\text{M} \times 100 \text{ B} = \mathbf{\sim 10 \text{ GB}}$ hot working set
- Replicated 1× for HA $\rightarrow \mathbf{\sim 20 \text{ GB}}$ across the Redis cluster
- TTL policy: every key expires when its window does, so the set self-prunes

Bandwidth

- Each Redis call: $\sim 200 \text{ B request} + \sim 100 \text{ B response} \approx \mathbf{300 \text{ B}}$
- Cluster bandwidth: $10\text{M} \times 300 \text{ B} = \mathbf{3 \text{ GB/s}} \approx 24 \text{ Gbps}$ east-west
- Shard out to 16+ Redis nodes; each node sees $\sim 1.5 \text{ Gbps}$ — comfortable

KEY

Capacity Recap

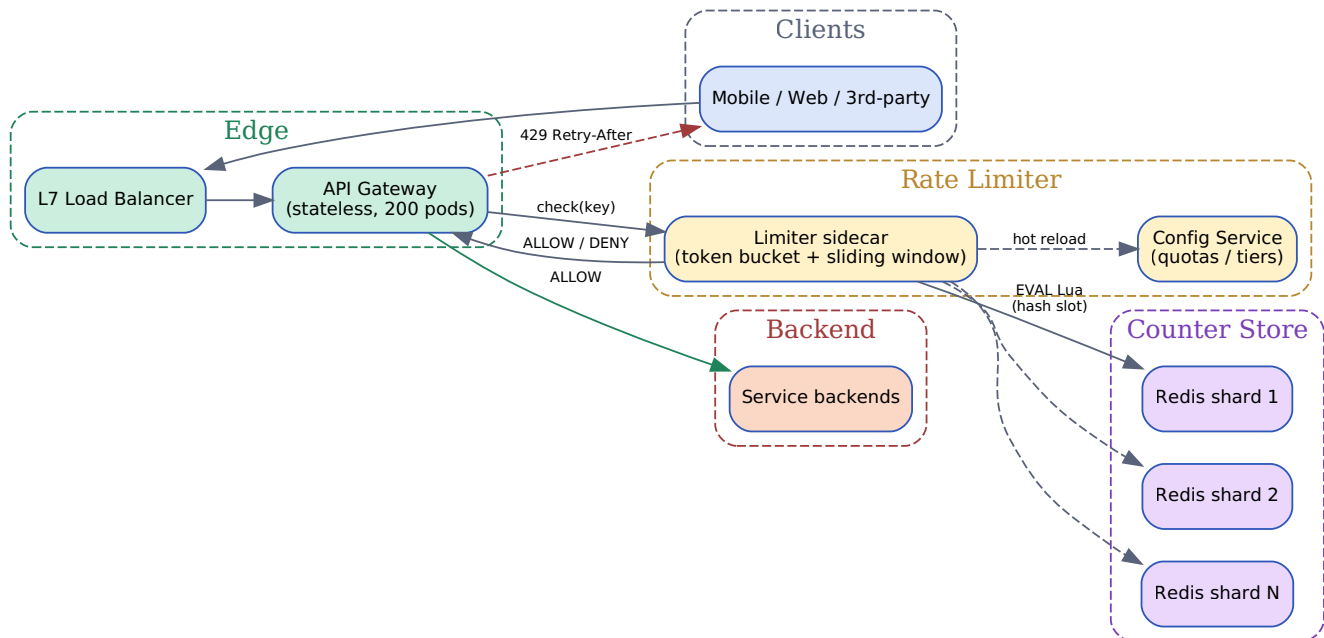
10M rps · 16 Redis shards · $\sim 625\text{K ops/shard}$ · $\sim 10 \text{ GB}$ hot state · $\text{p99} < 1 \text{ ms}$ on the gateway.

04

High-Level Architecture

Where the limiter sits and how it fans out

The limiter is a thin **library or sidecar** embedded in every API Gateway pod. On each request it looks up the identity, fetches the limit config, consults a sharded **Redis cluster** for the counter, and either passes the request to the backend or short-circuits with HTTP 429. The Redis cluster is the only piece of shared state — everything else is stateless.



Per-request flow: API Gateway → Rate Limiter (sidecar) → Redis cluster (sharded by key). On allow, the request continues to the backend; on deny, the gateway returns 429 with Retry-After.

Component Responsibilities

- **API Gateway**: terminates TLS, authenticates, extracts identity, calls limiter
- **Rate Limiter sidecar/library**: picks identity tier, builds keys, runs Lua check on Redis, attaches headers
- **Config Service**: publishes per-tier quotas; gateways subscribe over gRPC stream
- **Redis Cluster**: sharded by hash(key); holds counter state; replicated 1×
- **Backend**: receives only ALLOWED traffic; never sees abusers
- **Observability**: sample of allow/deny → Kafka → analytics for abuse detection

05

Algorithm Comparison

Five canonical rate-limiting algorithms

Five algorithms dominate the literature. Each makes a different trade between memory, accuracy, smoothness, and burst handling. The table below summarises what to pick when.

Side-by-Side

Algorithm	Accuracy	Memory / key	Burst	Pros	Cons
Fixed Window	Low	1 counter (~16 B)	2× at boundary	Trivial; one INCR per req	Edge bursts: 2× at the window flip
Sliding Window Log	Exact	O(N) timestamps (~KBs)	Smooth	Perfectly precise	Memory blows up under load
Sliding Window Counter	High (~1% err)	2 counters (~32 B)	Smooth	Cheap + accurate; weighted blend of two adjacent windows	Slight approximation; assumes uniform request distribution within a window
Token Bucket	High	2 numbers (~32 B)	Configurable burst (bucket size)	Smooth allowance + burst; intuitive	Two parameters to tune (rate, capacity)
Leaky Bucket	High	Queue (~KB)	None — strictly smoothed	Constant outflow; protects backends	Latency added by queue; harder to distribute

When to Pick Which

- **Token Bucket:** default for user-facing APIs — smooth steady rate plus tolerated bursts
- **Sliding Window Counter:** best when memory matters and you want near-exact accuracy
- **Fixed Window:** simple monthly/daily quotas where the boundary effect is acceptable
- **Sliding Window Log:** tiny per-user limits where exactness is required (e.g., login attempts)
- **Leaky Bucket:** when you must guarantee a steady rate to a fragile downstream

TIP

Recommended Default

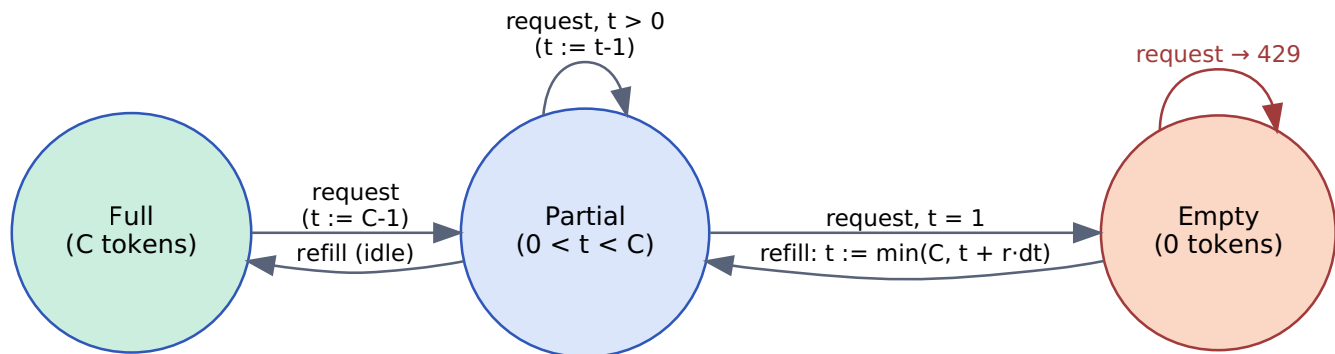
Use **Token Bucket** for the user-facing limit (smooth steady rate + configurable burst), and layer a **Sliding Window Counter** for the longer-window quotas (per-minute / per-day) where memory matters.

06

Token Bucket Deep Dive

Smooth rate + burst

A token bucket has two parameters: **capacity** (max tokens, i.e., burst size) and **refill rate** (tokens added per second). Each request takes one token; if the bucket is empty the request is denied. Refill is computed lazily, so we don't need a background ticker.



Token bucket state machine: tokens refill at rate r per second up to capacity C ; each request consumes a token. If empty, deny.

Reference Implementation (Pseudocode)

```

# Lazy token bucket: refill on demand, no background timer
# State per key: (tokens, last_refill_ts)
# Config:          capacity C, refill_rate r tokens/sec

def allow(key, C, r, now):
    tokens, last_ts = store.get(key, default=(C, now))
    elapsed = now - last_ts
    tokens = min(C, tokens + elapsed * r) # lazy refill
    if tokens >= 1:
        tokens -= 1
        store.set(key, (tokens, now), ttl=2*C/r)
        return ALLOW, remaining=tokens
    retry_after = (1 - tokens) / r # seconds until next token
    store.set(key, (tokens, now), ttl=2*C/r)
    return DENY, retry_after
  
```

Tuning

Tier	Steady rate (r)	Burst (C)	Window math
Free	10 rps	20 tokens	≈ 600 req/min sustained
Premium	100 rps	200 tokens	≈ 6,000 req/min sustained
Enterprise	1,000 rps	2,000 tokens	Custom; dedicated Redis shard

INSIGHT

Why lazy refill?

A background ticker for 100M keys is impossible. Computing refill on demand ($\text{elapsed} \times \text{rate}$) collapses the whole problem to a single read-modify-write per request — and Redis can do that atomically with a Lua script.

07

Sliding Window Counter

Accuracy at low memory

Sliding Window Counter solves the boundary problem of Fixed Window without the memory cost of Sliding Window Log. We keep two adjacent fixed-window counters (current + previous) and linearly interpolate based on how far into the current window we are.

The Math

Let W = window size (e.g., 60 sec), now = current time, and $elapsed = now \bmod W$. Define:

- $curr$ = count in the current window $[now - elapsed, now]$
- $prev$ = count in the previous window $[now - elapsed - W, now - elapsed]$
- $weight = (W - elapsed) / W$ — how much of the previous window still overlaps a sliding W -second view
- **$estimated_count = prev \times weight + curr$**
- Allow if $estimated_count < limit$, then **INCR** $curr$

Pseudocode

```
# Sliding Window Counter
# Two keys per quota: rl:{id}:{window_start} stores a single integer count

def allow(id, limit, W, now):
    curr_window = (now // W) * W
    prev_window = curr_window - W
    elapsed     = now - curr_window
    weight      = (W - elapsed) / W           # fraction of prev still in view

    curr = redis.get(f'rl:{id}:{curr_window}') or 0
    prev = redis.get(f'rl:{id}:{prev_window}') or 0
    estimate = prev * weight + curr

    if estimate >= limit:
        return DENY, retry_after = W - elapsed

    redis.incr(f'rl:{id}:{curr_window}')
    redis.expire(f'rl:{id}:{curr_window}', 2 * W)  # auto-prune
    return ALLOW, remaining = limit - (estimate + 1)
```

Accuracy

- Assumption: requests are **uniformly distributed** within the previous window
- Empirically the error is **~0.003%** under steady traffic and at most a few percent under bursts
- Memory: **2 integers per key** (~32 B) regardless of QPS
- Compare to Sliding Window Log: $O(N)$ timestamps — KB-MB per heavy user

INSIGHT

Cloudflare scale

Cloudflare published that Sliding Window Counter handled ~400 req/sec average with ~0.003% error in their production fleet — small enough to ignore, with memory bounded to two counters per key.

08

Distributed Implementation

Three strategies for sharing counter state

Once you have N stateless gateway pods, the counter must live somewhere they can all see. Three patterns dominate, each with different consistency / latency / cost trade-offs.

Three Patterns

Pattern	How it works	Pros	Cons
Centralized Redis	Every gateway calls the same Redis cluster on every request	Globally accurate; one source of truth; trivial to reason about	Adds ~0.3–0.7 ms RTT; Redis is on the critical path
Local + periodic sync	Each gateway holds its own bucket; pushes deltas to Redis every ~100 ms	Sub-100 μ s decisions; survives Redis hiccups	Brief over-allow during sync gap; quotas exceeded by ~N pods $\times$ tiny amount
Approximate (sketch)	Count-Min Sketch / HyperLogLog at the edge for huge fan-in	Fixed memory regardless of cardinality; fast	Probabilistic; only suits coarse limits (e.g., DDoS detection)

Recommendation

- **Default:** Centralized Redis with a Lua script for atomicity — accurate and simple
- **Hot keys / latency-sensitive paths:** add a per-pod local bucket as L1 with periodic Redis reconcile
- **DDoS / volumetric:** Count-Min Sketch in front of everything; only hits Redis if sketch crosses threshold

Atomic Check-and-Increment in Redis (Lua)

```
-- Token bucket as a Redis Lua script (atomic, ~50  $\mu$ s round-trip)
-- KEYS[1] = bucket key   e.g. 'rl:user:42:GET:/v1/charges:tb'
-- ARGV[1] = capacity C
-- ARGV[2] = refill rate (tokens / second)
-- ARGV[3] = current timestamp (ms)
-- ARGV[4] = requested tokens (default 1)

local capacity    = tonumber(ARGV[1])
local refill      = tonumber(ARGV[2])
local now_ms      = tonumber(ARGV[3])
local requested   = tonumber(ARGV[4] or '1')
```

```

local state = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens  = tonumber(state[1]) or capacity
local last_ms = tonumber(state[2]) or now_ms

-- lazy refill
local elapsed_ms = math.max(0, now_ms - last_ms)
tokens = math.min(capacity, tokens + (elapsed_ms * refill / 1000))

local allowed = tokens >= requested
if allowed then tokens = tokens - requested end

redis.call('HMSET', KEYS[1], 'tokens', tokens, 'ts', now_ms)
-- Auto-expire when bucket would be full again (idle window)
redis.call('PEXPIRE', KEYS[1], math.ceil(capacity * 1000 / refill))

return { allowed and 1 or 0, tokens, capacity }

```

TIP

Why Lua and not WATCH/MULTI

EVAL runs as a single atomic blob inside Redis — no round-trip per op, no optimistic-locking retry storms. At 10M rps, the cost of optimistic retries would dwarf everything else.

09

Identity Hierarchy & Tiers

Who is the caller, and what are they allowed?

Each incoming request matches the tightest applicable bucket. We try identity in order — API key first (most specific) down to IP (least specific) — and rate-limit by all that apply. A single request may consume from multiple buckets in parallel.

Identity Hierarchy

Tier	Resolved from	Why we use it	Caveat
API key	Authorization: Bearer ...	Most specific; ties to billable account	Useless for unauthenticated traffic
user_id	JWT subject claim	Cross-device per-human limit	Requires login session
device fingerprint	Cookie / header / SDK	Catches anonymous client abuse	Spoofable; combine with IP
IP address	X-Forwarded-For (trusted hops only)	Last resort; covers fully anonymous	NAT shares an IP across many users

Tiered Quotas

Plan	Per-second	Per-minute	Per-day	Burst
Anonymous (IP)	10 rps	300 / min	10K / day	20
Free	20 rps	1K / min	100K / day	40
Premium	100 rps	5K / min	1M / day	200
Enterprise	1K rps	30K / min	10M / day	2K (custom)
Internal	—	—	—	Bypass list

Resolution Algorithm

1. Extract identity (API key > JWT > device > IP)
2. Look up tier in Config Service (cached locally for 5 sec)
3. Build keys: `rl:<tier>:<id>:<endpoint>:<window>`

4. EVAL Lua atomically against each window (per-second + per-minute + per-day)
5. DENY if *any* bucket says deny; ALLOW only if *all* say allow
6. Set headers: X-RateLimit-Limit/Remaining/Reset, Retry-After

WARNING

Don't trust raw IPs

Always pull the IP from the trusted edge proxy (Cloudflare/ALB) — never the client-supplied X-Forwarded-For. Otherwise a bad actor crafts the header and drains other users' buckets.

10

Persistence Schema

What we store and where

Two stores: a **Redis cluster** for hot counter state, and a **Postgres rate_limits table** as the durable mirror of configuration and slow-changing per-account overrides.

Redis Counter Layout

```
# Token bucket state – Redis hash
rl:tb:{tier}:{id}:{endpoint}
  HMSET
    tokens    <float>      # current tokens remaining
    ts        <ms>         # last refill timestamp
  PEXPIRE <ms>              # auto-prune after a full idle window

# Sliding-window counter – two integers
rl:swc:{tier}:{id}:{endpoint}:{window_start}
  INCR ...                  # one integer per fixed window
  EXPIRE 2*W                # keep current and previous only

# Hot-key shadow set for analytics
rl:topn  ZSET                # top abusers by deny count, last 60 sec
```

Postgres rate_limits Table

```
CREATE TABLE rate_limits (
  tier                VARCHAR(32) NOT NULL,           -- free/premium/enterprise
  endpoint_pattern    VARCHAR(128) NOT NULL,          -- e.g. 'GET /v1/charges'
  window_seconds      INT NOT NULL,                  -- 1, 60, 86400
  max_requests        INT NOT NULL,                   -- limit per window
  burst_capacity       INT,                           -- token-bucket capacity
  refill_per_sec       NUMERIC(10,2),
  algorithm           VARCHAR(16) DEFAULT 'token_bucket',
  enabled             BOOLEAN DEFAULT TRUE,
  updated_at          TIMESTAMP DEFAULT NOW(),
  PRIMARY KEY (tier, endpoint_pattern, window_seconds)
);

CREATE TABLE rate_limit_overrides (
  account_id         BIGINT NOT NULL,
  endpoint_pattern    VARCHAR(128) NOT NULL,
  window_seconds      INT NOT NULL,
  max_requests        INT NOT NULL,
  burst_capacity       INT,
```

```
    expires_at      TIMESTAMP,          -- temporary boost
    reason          VARCHAR(256),       -- 'support ticket #1234'
    PRIMARY KEY (account_id, endpoint_pattern, window_seconds)
);

CREATE INDEX idx_overrides_acct ON rate_limit_overrides (account_id);
```

Config Propagation

- Config Service polls Postgres every 1 sec; pushes deltas over gRPC stream
- Gateways cache the active config in process memory; refresh on push or 5 sec poll
- Schema versioning: every config has a `version`; gateways report which version they're running
- Rollout safety: changes deploy as canary tier first, then graduated rollout

11

Hot Keys & Sharding

Surviving viral abusers and Black-Friday users

The Hot Key Problem

When one API key gets hammered (a misconfigured client, a viral integration), all the load lands on one Redis shard. At 10M rps cluster-wide, even a tiny fraction of a single user can saturate a single shard.

Mitigations

Technique	How it works	When to use
Key salting	Add <code>:shard<0..K></code> suffix; pick suffix at random; merge counters across	Single key sees > 10K rps
Local L1 bucket	Per-pod token bucket; only call Redis on near-empty	Latency-sensitive paths
Probabilistic admission	Below a threshold, allow without checking Redis (sample 1 in N)	Below 10% of limit
Hot-key escape hatch	Detect via Redis cluster's hot-key tracker; promote to dedicated shard	Persistent abusive keys
Circuit breaker	If Redis p99 > 5 ms, fall back to local-only for 30 sec	Redis under stress

Shard Topology

- Redis Cluster mode: **16 primary shards**, each with 1 replica
- Hash slot = $\text{CRC16}(\text{key}) \% 16384$; client library routes directly to the right node
- Hot key salting: `rl:tb:premium:42:GET:/v1/charges:s7` for K=16 sub-buckets
- Resharding: add nodes online; cluster reshuffles slots without downtime

TIP

Don't oversalt

Salting trades accuracy for spread. With K=16 sub-buckets the effective burst is K× the configured one. Apply salting only to keys that actually hot-spot; default to no salting.

12

Failure Modes

What breaks and how we degrade

Failure	Impact	Detection	Mitigation
Redis primary down	Counter writes fail; deny path uncertain	Cluster gossip + client error rate	Auto-failover to replica (~5–15 sec); meanwhile fail-open for reads, fail-closed for writes
Whole Redis cluster outage	All buckets unavailable	Cluster client errors > 50%	Circuit-breaker trips → local-only mode (per-pod best-effort) for 30 sec, then retry
Network partition gateway ↔ Redis	Some pods see Redis, others don't	Per-pod success rate divergence	Each pod independently decides fail-open/closed by policy table
Lua script bug	Allow-all or deny-all on one shard	Anomaly detection on allow ratio	Rollback config; scripts shipped through canary
Config Service down	No new quota changes propagate	Heartbeat from gateway fleet	Gateways keep last-known config in process memory
Hot key saturating one shard	p99 latency spike for that shard	Per-shard p99 + hot-key tracker	Auto-salt key; promote to dedicated shard; throttle abuser explicitly
Clock skew between pods	Sliding-window weights drift	NTP sync alarms	Use Redis time (TIME command) as canonical; reject if local-Redis $\Delta > 500$ ms
Gateway pod crash	Loses local L1 bucket state	Pod restart event	Stateless: counter lives in Redis; new pod re-derives L1 lazily

Fail-Open vs Fail-Closed

Endpoint class	Policy on Redis outage	Why
Idempotent reads (GET /v1/products)	Fail-open	Better to serve a few extra reads than 5xx on outage
Login / auth attempts	Fail-closed	Brute-force prevention is the whole point
Write traffic / billing	Fail-closed	Loss of accuracy could cost real money

Endpoint class	Policy on Redis outage	Why
DDoS-edge limits	Fail-closed	If we can't measure, we must be conservative

WARNING

Fail-open isn't free

Fail-open during a Redis outage means real abusers can briefly bypass the limiter. Always pair fail-open with a per-pod fallback bucket — even an approximate local limit is better than no limit.

13

Observability & Abuse Detection

Watching the watcher

Core Metrics

Metric	Why we care	Alert threshold
allow_ratio	Is the limiter working at all?	Drops < 99% suddenly → bug or DDoS
redis_p99_ms	Critical-path latency	> 1 ms p99 sustained
hot_key_qps	Spot abusers / misconfigured clients	Single key > 10K rps for 60 sec
fail_open_seconds	Reliability of the limiter itself	> 30 s/day
config_age_seconds	Stale config propagation	> 30 s on any pod
cluster_top_denies	Top-N denied (key, endpoint) pairs	Track for security review

Pipeline

- Per-request: emit {key, endpoint, allow, latency} at 1% sample to Kafka
- Stream processor (Flink / Kafka Streams): aggregate per-key per-minute denies
- Top-N writer: HLL + count-min sketch for cardinality + top-K abusers
- Anomaly detector: z-score on allow_ratio per endpoint; auto-page if $> 3\sigma$
- Long-term: roll up to Postgres for billing-style reporting (per account, per day)

INSIGHT

Abuse loop

A good limiter is also a good abuse detector: every **429** is a data point. Top-N abusers feed back into automated bans, CAPTCHAs, or — in extreme cases — Cloudflare WAF rules at the edge.

14

Design Trade-offs

Why we made each choice

Decision	Choice	Trade-off
Algorithm	Token bucket + sliding window counter	Smooth burst-friendly user limit; cheap accurate long-window. Pure Fixed Window is simpler but allows 2× edge bursts.
Counter store	Centralized Redis cluster	Globally accurate at 10M rps. Local-only would be sub-100 μs but allow N× over-quota where N = pod count.
Atomicity	Lua EVAL	One round-trip; atomic. WATCH/MULTI is portable but blows up with retries under contention.
Failure policy	Mixed (open for reads, closed for writes)	Maximises availability without sacrificing money-paths. Always-closed sacrifices uptime; always-open sacrifices safety.
Identity	API key > user > device > IP	Most specific wins; covers anonymous + authenticated. IP-only is unfair to NAT users.
Tiered quotas	Free / Premium / Enterprise + override table	Simple defaults plus surgical overrides. Per-account-only is hard to manage at scale.
Distribution model	Centralized + L1 fallback	Accuracy by default; latency win on hot keys. Always-local would over-allow; always-central adds RTT.
Sharding	Redis Cluster, hash by key	Even spread for most keys; salting for hot ones. Fixed sharding by user_id concentrates load on whales.
Config propagation	Push via gRPC stream	<5 s convergence. Pull every N sec is simpler but slower to react to incident-time changes.
Observability	1% sample to Kafka	Cheap, statistically meaningful. Full sampling is 10M events/sec — too expensive.

TIP

Latency budget

Of our < 1 ms target: ~0.4 ms Redis RTT (same-AZ), ~0.2 ms Lua execution, ~0.2 ms gateway overhead, ~0.2 ms slack. If the Redis hop costs more than 0.6 ms p99, move to a closer cell or drop to L1-only for that endpoint.

15

Interview Playbook

How to walk through this in 45 minutes

Rate limiter is one of the most interview-friendly system-design questions: the scope is tractable, the algorithms are crisp, and there's a clear consistency-vs-latency trade-off to debate. Use the outline below to keep the conversation tight.

45-Minute Outline

1. **Clarify (3 min):** who's the caller, where does the limiter sit, hard or soft, single or multi-window
2. **Capacity (4 min):** 10M rps, 100M keys, ~10 GB Redis, p99 < 1 ms
3. **Algorithm choice (6 min):** walk the table; pick token bucket + sliding window counter; defend why not Fixed/Log
4. **High-level architecture (4 min):** sidecar limiter, Redis cluster, config service, fail policy
5. **Lua deep-dive (6 min):** show the script; explain atomicity; why not WATCH/MULTI
6. **Hot keys (4 min):** salting, L1 buckets, dedicated shards
7. **Failure modes (5 min):** Redis outage, hot key, clock skew; per-endpoint open/closed policy
8. **Identity & tiers (5 min):** hierarchy, override table, header contract
9. **Trade-offs & follow-ups (8 min):** defend choices; entertain extensions (global, ML-based, GDPR)

Common Follow-ups

- **Q: Why not just count in memory per pod?** A: With 200 pods, an effective limit of 1K rps becomes 200K rps; centralized counter is the only way to be globally accurate.
- **Q: How do you handle a Redis outage?** A: Per-endpoint policy: read endpoints fail-open with a per-pod best-effort bucket; auth and write endpoints fail-closed.
- **Q: Token bucket vs leaky bucket?** A: Token bucket allows bursts; leaky bucket smooths to a constant rate. For user-facing APIs, bursts are nice; for fragile downstreams, leaky.
- **Q: How do you stop a single key from saturating one shard?** A: Salt the key into K sub-buckets, sum the counters atomically. Trade slight burst inflation for shard spread.
- **Q: What about clock skew?** A: Use Redis TIME as the canonical clock inside Lua; reject the request if local-Redis skew > 500 ms.
- **Q: Sliding window log accuracy is exact — why not?** A: O(N) memory per key. At 100M keys with thousands of timestamps each, that's TBs of state. Counter approach is ~32 B/key.
- **Q: Global rate limit across regions?** A: Either a single global Redis (high RTT) or eventual reconciliation across regional clusters (slight over-allow). Most use regional + slow merge.

KEY

Memorize These Numbers

10M req/sec · **p99 < 1 ms** · **~100M keys** · **~10 GB Redis** (16 shards, ~625K ops/shard) · **~32 B / key**
for sliding window counter · **~50 μ s** Lua eval · **0.003% SWC error** · Free 20 rps / Premium 100 rps /
Enterprise 1K rps · fail-open reads / fail-closed writes.